

20250731日报

今日工作内容

1. vtprotobuf优化方法完成现网测试，并和使用proto标准库的反序列化方法进行对比。

使用codec.RegisterSerializer(codec.SerializationTypePB, vtprotobuf.NewVTSerializer())
覆盖trpc-go中对于pb文件的反序列化和序列化函数。首先在本地对比Unmarshal()和
UnmarshalVT()：

```
11 func Benchmark_1(b *testing.B) {
12     data, _ := os.ReadFile("../output.bin")
13     for n := 0; n < b.N; n++ {
14         req := &fc_mall_feature.TransformRequest{}
15         if err := proto.Unmarshal(data, req); err != nil {
16             b.Fatal(err)
17         }
18     }
19 }
20
21 run benchmark | debug benchmark
22 func Benchmark_2(b *testing.B) {
23     data, _ := os.ReadFile("../output.bin")
24     for n := 0; n < b.N; n++ {
25         req := &fc_mall_feature.TransformRequest{}
26         if err := req.UnmarshalVT(data); err != nil {
27             b.Fatal(err)
28         }
29     }
30 }
31 // func Benchmark_3(b *testing.B) {
32 //     data, _ := os.ReadFile("output.json")
33 //     for n := 0; n < b.N; n++ {
34 //         req := &fc_mall_feature.TransformRequest{}
35 //         if err := req.UnmarshalVT(data); err != nil {
36 //             b.Fatal(err)
37 //         }
38 //     }
39 // }
```

问题 20 输出 调试控制台 终端 窗口 9

```
• [root@drewjjli-1t70osepa fc_mall_feature]# go test -bench=. -benchmem ./test
goos: linux
goarch: amd64
pkg: git.woa.com/drewjjli/helloworld/sgameproto/full_control/fc_mall_feature/test
cpu: Intel(R) Xeon(R) Platinum 8255C CPU @ 2.50 GHz
Benchmark_1-48          217          5158975 ns/op          1159684 B/op          22886 allocs/op
Benchmark_2-48          572          2095023 ns/op           982459 B/op          14317 allocs/op
PASS
ok      git.woa.com/drewjjli/helloworld/sgameproto/full_control/fc_mall_feature/test    3.125s
```

从结果可看出，UnmarshalVT () 大概比proto.Unmarshal()快了2.4倍左右。

proto标准库性能差的主要原因是proto的序列化和反序列化都利用了反射，如：

```

// Marshal returns the wire-format encoding of m.
func Marshal(m Message) ([]byte, error) {
    b, err := marshalAppend(nil, m, false)
    if b == nil {
        b = zeroBytes
    }
    return b, err
}

var zeroBytes = make([]byte, 0, 0)

func marshalAppend(buf []byte, m Message, deterministic bool) ([]byte, error) {
    if m == nil {
        return nil, ErrNil
    }
    mi := MessageV2(m)
    nbuf, err := protoV2.MarshalOptions{
        Deterministic: deterministic,
        AllowPartial: true,
    }.MarshalAppend(buf, mi)
    if err != nil {
        return buf, err
    }
    if len(buf) == len(nbuf) {
        if !mi.ProtoReflect().IsValid() {
            return buf, ErrNil
        }
    }
    return nbuf, checkRequiredNotSet(mi)
}

```

Marshal()中的MessageV2()就使用了反射机制reflect.ValueOf(m)。

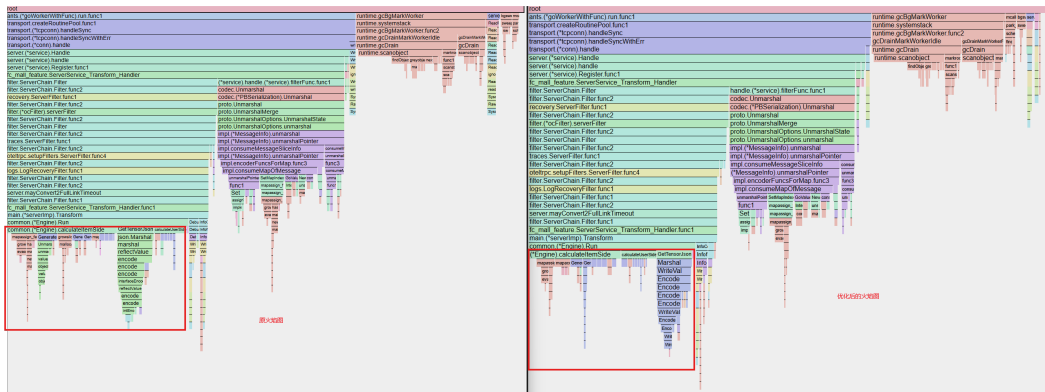
目前针对fc_mall_feature性能优化总结如下：

方法：

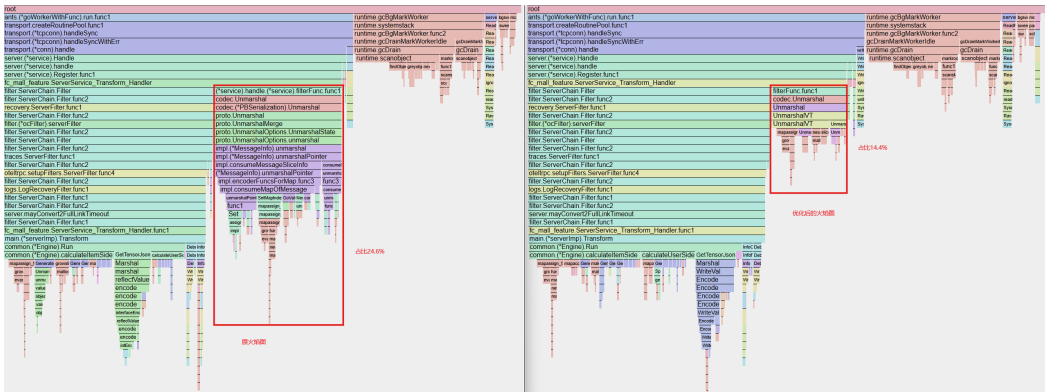
- 1、针对calculateItemSide()和calculateUserSide()函数中的mapassign_faststr和growslice修改：预分配大小；
- 2、对于GetTensorJson()和HotSaleGenerator.Generate()中的json.Unmarshal和json.Marshal，可以采用codec.JSONAPI来进行优化，即改为codec.JSONAPI.Unmarshal和codec.JSONAPI.Marshal；
- 3、替代传统的proto.Unmarshal()，使用vtprotobuf结合protoc进行辅助编译，即使用UnmarshalVT()代替proto.Unmarshal()来解析字节流；

结果对比如下：

1. 使用1，2点手段得到的火焰图和原火焰图几乎无差别，占比影响不大



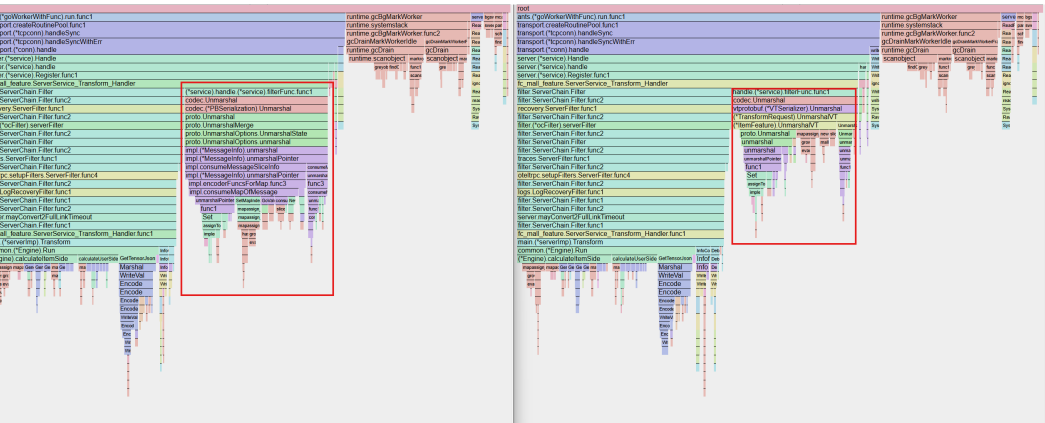
2. 在1, 2手段的基础上，使用3后得到的结果如下：



利用vtprotobuf后pb的解析占比减少了10%左右。

今日所遇问题：

1. 问题：今天第一次现网测试vtprotobuf效果的时候，几乎看不出来影响，其结果如下：



原因：此时我只是把fc_mall_feature外层替换成了UnmarshalVT(), 真正的大头数据是serving.Value{}, 因此需要对trpc.sgamesvr.feature_platform也要用vtprotobuf来编译。

明日待办：

1. 在fc_mall_feature服务上的测试环境和体验服环境分别测试七彩石容灾效果；

2. 推进fc_mall_feature性能优化的完成；